

```

In [ ]: import newReader
import glob
import gzip
import pandas as pd
import numpy as np
import os
from datetime import datetime, timedelta
import pytz
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
pd.options.mode.copy_on_write = True
%matplotlib inline
# Correcting the directory path
raw_dir = "/Users/SEELMPC1/Desktop/solarrawz/1s/20240408" #"/Users/SEELMPC1/
#raw_dir = "/Users/SEELMPC1/Desktop/EZIE-Mag_Testing_Git/ezie-mag-testing/EZ
# Updated pattern to match files, ensuring all potential subdirectories are
raw_filepaths = glob.glob(os.path.join(raw_dir, "**", "*.rawz"), recursive=True)

print(f"Number of files found: {len(raw_filepaths)}") # Debugging line to c

data_records = [] # List to hold all records

utc = pytz.utc

EDT_OFFSET = timedelta(hours=4)
for path in raw_filepaths:
    print(f"Processing file: {path}")
    try:
        with gzip.open(path, "r") as gzf:
            while True:
                rec = newReader.readrawz(gzf)
                if rec is None:
                    break
                #print(rec['tval'])
                # Adjust for the EDT offset to convert to actual UTC
                rec_time = (datetime.fromtimestamp(rec['tval']) + EDT_OFFSET)

                # Store the UTC datetime in the record
                rec['dateTime'] = rec_time
                data_records.append(rec)

    except Exception as e:
        print(f"Error processing {path}: {e}")

# Create DataFrame
df = pd.DataFrame(data_records)

# Set the index as the dateTime column
df.set_index('dateTime', inplace=True)

df['B_mean'] = [np.sqrt(x**2 + y**2 + z**2) for x, y, z in zip(df['BxnT'], c

```

Number of files found: 3

Processing file: /Users/SEELMPC1/Desktop/solarrawz/1s/20240408/eziemag.2024040814.AAAAAA0PdM8A.1s.rawz

Processing file: /Users/SEELMPC1/Desktop/solarrawz/1s/20240408/eziemag.2024040815.AAAAAA0PdM8A.1s.rawz

Processing file: /Users/SEELMPC1/Desktop/solarrawz/1s/20240408/eziemag.2024040813.AAAAAA0PdM8A.1s.rawz

Non-Timezone Adjusted UNIX Date Time Readings (EDT):

File **13 Calibration Phase:

Monday, April 8, 2024 1:54:55.499 PM

Monday, April 8, 2024 1:57:27.270 PM

File **14: Partial Eclipse Begin

Monday, April 8, 2024 2:00:00.590 PM

Monday, April 8, 2024 2:59:59.210 PM

File **15 (100% Confidence in Accuracy):

Monday, April 8, 2024 3:00:00.235 PM

Monday, April 8, 2024 3:22:46.893 PM

Eclipse Overview:

- A total eclipse will begin at 1:59 p.m. and end at 4:29 p.m.
- Totality begins at 3:13 p.m. and ends at 3:17 p.m.

```
In [ ]: import numpy as np
import logging
from scipy.stats import t
from scipy.optimize import curve_fit

#-----Plotting Functions-----#
def prediction_band(x, cov, params):
    # Calculate the prediction band using the covariance matrix
    # x: array of x values
    # cov: covariance matrix of the fit parameters
    # params: dictionary of fit parameters

    # Ensure that x is a numpy array
    x = np.array(x)

    # Calculate the variance of the prediction for each x value
```

```

# The variance is a sum of variances and covariances for the linear model
#  $\text{var}(y) = (dA)^2 * x^2 + (dB)^2 + 2 * x * \text{cov}(A,B)$ 
A_var = cov[0, 0]
B_var = cov[1, 1]
cov_AB = cov[0, 1]

# Calculate the variance for each x point
y_var = A_var * x**2 + B_var + 2 * x * cov_AB

# Standard error is the square root of the variance
y_err = np.sqrt(y_var)

return y_err

#-----Weighted Average-----#
def calcWeightedAverage(x, dx):
    x = np.asarray(x)
    dx = np.asarray(dx)
    inv_dx_squared = np.power(dx, -2)

    wav = np.sum(inv_dx_squared * x) / np.sum(inv_dx_squared)
    d_wav = 1 / np.sqrt(np.sum(inv_dx_squared))
    return wav, d_wav

#-----Significance-----#
def significance(x1, x2, delta_x1, delta_x2):
    """Function to calculate the significance of the difference between two

    Args:
        x1 (numpy array): 1st var
        x2 (numpy array): 2nd var
        delta_x1 (numpy array): 1st var uncertainty
        delta_x2 (numpy array): 2nd var uncertainty

    Returns:
        float: Significance of the difference between two values
    """
    return np.abs((x1 - x2) / np.sqrt(delta_x1**2 + delta_x2**2))

#-----Fancy Fitting-----#
# Define fitting functions and their derivatives

# Configure logging to help with debugging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s

def linear_fit(x, A, B):
    return A * x + B

def derivative_linear_fit(x, A, B):
    return A

def power_fit(d, A, B):
    return A * d ** B

```

```

def derivative_power_fit(d, A, B):
    return B * A * d ** (B - 1)

def log_fit(x, A, B):
    return A * np.log(x) + B

def derivative_log_fit(x, A, B):
    if np.any(x <= 0):
        raise ValueError("x for log_fit must be greater than zero")
    return A / x

# Function to select the appropriate derivative function
def select_derivative_method(method):
    if method == power_fit:
        return derivative_power_fit
    elif method == log_fit:
        return derivative_log_fit
    elif method == linear_fit:
        return derivative_linear_fit
    else:
        raise ValueError("Selected method does not have a defined derivative")

# Function to iteratively refine parameters with error propagation
# Select the method here to switch between fitting functions
method = linear_fit # Change to power_fit as needed

def get_params(x, y, xerr, yerr, method, tol, max_iterations=10):
    """
    Iteratively refines the parameters of a specified fitting method to model the data.
    Error propagation is taken into account by adjusting the weights (effective sample size).

    Parameters:
    x (array_like): Independent data points.
    y (array_like): Dependent data points that are to be fitted.
    xerr (array_like): Uncertainties in the independent data points.
    yerr (array_like): Uncertainties in the dependent data points.
    method (function): The fitting method to be used (e.g., log_fit, power_fit, linear_fit).
    max_iterations (int): The maximum number of iterations to perform for refinement.
    tol (float): Tolerance to try and reach.

    Returns:
    dict: A dictionary containing the refined parameters 'A' and 'B', and the covariance matrix.
    array_like: The covariance matrix of the fitted parameters.

    Raises:
    RuntimeError: If curve fitting fails to converge within the maximum number of iterations.

    Notes:
    The function uses the scipy.optimize.curve_fit method for fitting and iteratively refines the parameters.
    It updates the initial parameters 'p0' at each iteration with the latest refined parameters.

    Example usage:
    """

```

```

params, cov = get_params(data['Distance (cm)'], data["Potential (Volts)"]
                        np.full(len(data), distance_uncertainty), data[
                        method=log_fit)
print("Parameters:", params, "\nCovariance Matrix:", cov)
"""

# Ensure all x values are greater than zero for log_fit
if method == log_fit and np.any(x <= 0):
    raise ValueError("All x values must be greater than zero for logarithmic fit")

initial_A = np.polyfit(x, y, 1)[0] # Slope
initial_B = np.polyfit(x, y, 1)[1] # Intercept

# Now use these as your initial parameters
p0 = [initial_A, initial_B]
iteration = 0
effective_sigma = yerr
old_params = None

while iteration < max_iterations:
    try:
        # Ensure uncertainties are not zero to avoid divide by zero
        xerr = np.maximum(xerr, np.full_like(xerr, np.finfo(float).eps))
        yerr = np.maximum(yerr, np.full_like(yerr, np.finfo(float).eps))

        # Perform curve fitting
        p, cov = curve_fit(method, x, y, sigma=effective_sigma, absolute_sigma=True)
    except RuntimeError as e:
        logging.warning(f"Curve fitting failed: {e}")
        break

    new_params = {"A": p[0], "dA": np.sqrt(cov[0,0]), "B": p[1], "dB": np.sqrt(cov[1,1])}
    logging.info(f"Iteration {iteration}: {new_params}")

    # Check for convergence
    if old_params:
        differences = [abs(new_params[key] - old_params[key]) / (old_params[key] + 1e-10) for key in new_params]
        if max(differences) <= tol:
            logging.info("Convergence reached.")
            break

    old_params = new_params.copy()
    derivative_method = select_derivative_method(method)
    derivative = derivative_method(x, *p)
    effective_sigma = np.sqrt(np.maximum(yerr**2 + (derivative * xerr)**2, 1e-10))
    p0 = [p[0], p[1]] # Update the initial guess with the latest fit parameters
    iteration += 1

return new_params, cov

```

```

In [ ]: import numpy as np
import logging
from scipy.stats import t
from scipy.optimize import curve_fit

```

```

#-----Plotting Functions-----#
def prediction_band(x, cov, params):
    # Calculate the prediction band using the covariance matrix
    # x: array of x values
    # cov: covariance matrix of the fit parameters
    # params: dictionary of fit parameters

    # Ensure that x is a numpy array
    x = np.array(x)

    # Calculate the variance of the prediction for each x value
    # The variance is a sum of variances and covariances for the linear model
    #  $\text{var}(y) = (dA)^2 * x^2 + (dB)^2 + 2 * x * \text{cov}(A,B)$ 
    A_var = cov[0, 0]
    B_var = cov[1, 1]
    cov_AB = cov[0, 1]

    # Calculate the variance for each x point
    y_var = A_var * x**2 + B_var + 2 * x * cov_AB

    # Standard error is the square root of the variance
    y_err = np.sqrt(y_var)

    return y_err

#-----Weighted Average-----#
def calcWeightedAverage(x, dx):
    x = np.asarray(x)
    dx = np.asarray(dx)
    inv_dx_squared = np.power(dx, -2)

    wav = np.sum(inv_dx_squared * x) / np.sum(inv_dx_squared)
    d_wav = 1 / np.sqrt(np.sum(inv_dx_squared))
    return wav, d_wav

#-----Significance-----#
def significance(x1, x2, delta_x1, delta_x2):
    """Function to calculate the significance of the difference between two

    Args:
        x1 (numpy array): 1st var
        x2 (numpy array): 2nd var
        delta_x1 (numpy array): 1st var uncertainty
        delta_x2 (numpy array): 2nd var uncertainty

    Returns:
        float: Significance of the difference between two values
    """
    return np.abs((x1 - x2) / np.sqrt(delta_x1**2 + delta_x2**2))

#-----Fancy Fitting-----#
# Define fitting functions and their derivatives

```

```

# Configure logging to help with debugging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s

def linear_fit(x, A, B):
    return A * x + B

def derivative_linear_fit(x, A, B):
    return A

def power_fit(d, A, B):
    return A * d ** B

def derivative_power_fit(d, A, B):
    return B * A * d ** (B - 1)

def log_fit(x, A, B):
    return A * np.log(x) + B

def derivative_log_fit(x, A, B):
    if np.any(x <= 0):
        raise ValueError("x for log_fit must be greater than zero")
    return A / x

# Function to select the appropriate derivative function
def select_derivative_method(method):
    if method == power_fit:
        return derivative_power_fit
    elif method == log_fit:
        return derivative_log_fit
    elif method == linear_fit:
        return derivative_linear_fit
    else:
        raise ValueError("Selected method does not have a defined derivative")

# Function to iteratively refine parameters with error propagation
# Select the method here to switch between fitting functions
method = linear_fit # Change to power_fit as needed

def get_params(x, y, xerr, yerr, method, tol, max_iterations=10):
    """
    Iteratively refines the parameters of a specified fitting method to model the data.
    Error propagation is taken into account by adjusting the weights (effectively 1/error^2).

    Parameters:
    x (array_like): Independent data points.
    y (array_like): Dependent data points that are to be fitted.
    xerr (array_like): Uncertainties in the independent data points.
    yerr (array_like): Uncertainties in the dependent data points.
    method (function): The fitting method to be used (e.g., log_fit, power_fit, linear_fit).
    max_iterations (int): The maximum number of iterations to perform for refinement.
    tol (float): Tolerance to try and reach.

    Returns:
    """

```

dict: A dictionary containing the refined parameters 'A' and 'B', and the
array_like: The covariance matrix of the fitted parameters.

Raises:

RuntimeError: If curve fitting fails to converge within the maximum number of iterations.

Notes:

The function uses the `scipy.optimize.curve_fit` method for fitting and iteratively updates the initial parameters 'p0' at each iteration with the latest parameters.

Example usage:

```
params, cov = get_params(data['Distance (cm)'], data["Potential (Volts)"],
                        np.full(len(data), distance_uncertainty), data['Distance (cm)'],
                        method=log_fit)

print("Parameters:", params, "\nCovariance Matrix:", cov)
"""

# Ensure all x values are greater than zero for log_fit
if method == log_fit and np.any(x <= 0):
    raise ValueError("All x values must be greater than zero for logarithmic fit")

initial_A = np.polyfit(x, y, 1)[0] # Slope
initial_B = np.polyfit(x, y, 1)[1] # Intercept

# Now use these as your initial parameters
p0 = [initial_A, initial_B]
iteration = 0
effective_sigma = yerr
old_params = None

while iteration < max_iterations:
    try:
        # Ensure uncertainties are not zero to avoid divide by zero
        xerr = np.maximum(xerr, np.full_like(xerr, np.finfo(float).eps))
        yerr = np.maximum(yerr, np.full_like(yerr, np.finfo(float).eps))

        # Perform curve fitting
        p, cov = curve_fit(method, x, y, sigma=effective_sigma, absolute_sigma=True, p0=p0)

    except RuntimeError as e:
        logging.warning(f"Curve fitting failed: {e}")
        break

    new_params = {"A": p[0], "dA": np.sqrt(cov[0,0]), "B": p[1], "dB": np.sqrt(cov[1,1])}
    logging.info(f"Iteration {iteration}: {new_params}")

    # Check for convergence
    if old_params:
        differences = [abs(new_params[key] - old_params[key]) / (old_params[key] + 1e-10) for key in new_params]
        if max(differences) <= tol:
            logging.info("Convergence reached.")
            break

    old_params = new_params.copy()
    derivative_method = select_derivative_method(method)
    derivative = derivative_method(x, *p)
```



```

        effective_sigma = np.sqrt(np.maximum(yerr**2 + (derivative * xerr)**2, 1e-10))
        p0 = [p[0], p[1]] # Update the initial guess with the latest fit parameters
        iteration += 1

    return new_params, cov

```

Initial Plot

```

In [ ]: import matplotlib.pyplot as plt
import matplotlib.dates as mdates

# Plotting Temperature ('ctemp') over time
fig, ax1 = plt.subplots(figsize=(10, 6))

color = 'tab:red'
ax1.set_xlabel('Time on April 8, 2024 (UTC)')
ax1.set_ylabel('Temperature (°C)', color=color)
ax1.plot(df.index, df['ctemp'], color=color)
ax1.tick_params(axis='y', labelcolor=color)

# Instantiate a second y-axis to share the same x-axis
ax2 = ax1.twinx()

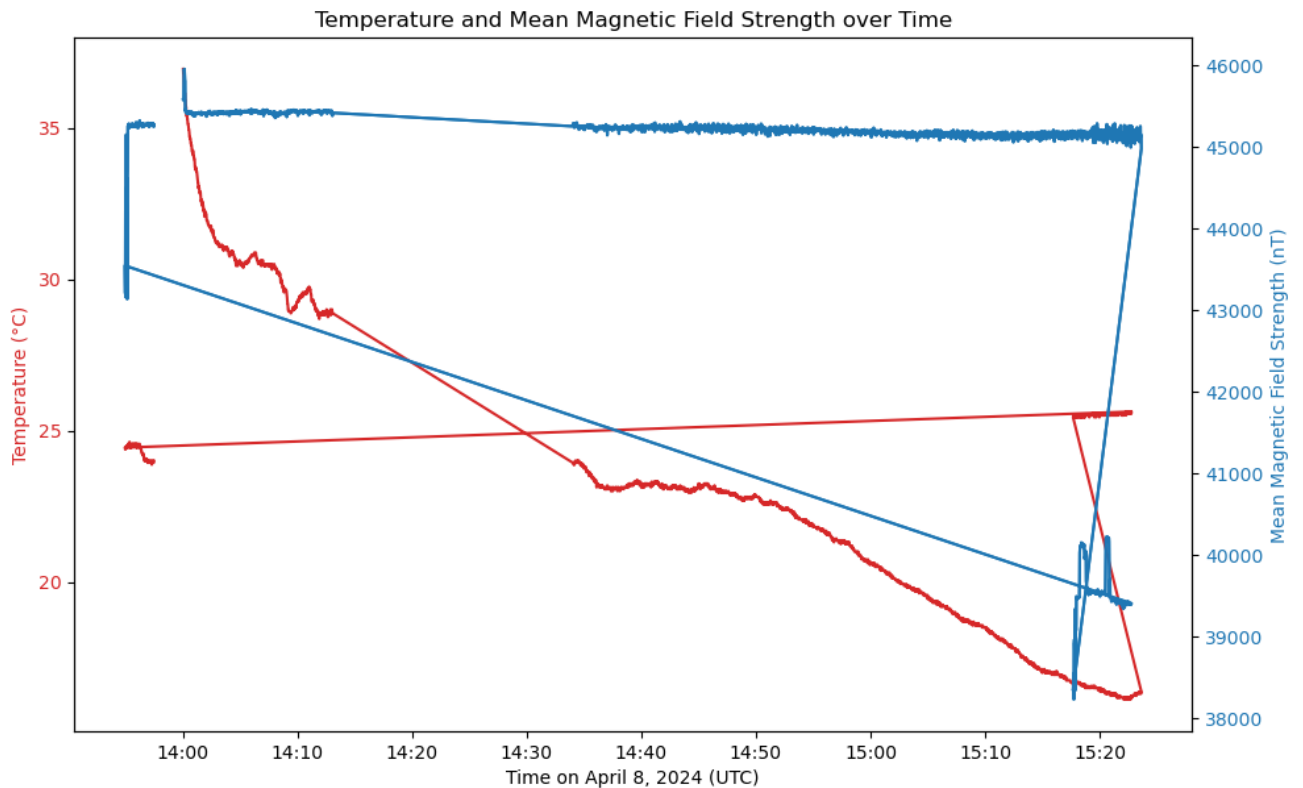
color = 'tab:blue'
ax2.set_ylabel('Mean Magnetic Field Strength (nT)', color=color) # we already have a label for this axis
ax2.plot(df.index, df['B_mean'], color=color)
ax2.tick_params(axis='y', labelcolor=color)

color = 'tab:blue'
ax2.set_ylabel('Mean Magnetic Field Strength (nT)', color=color) # we already have a label for this axis
ax2.plot(df.index, df['B_mean'], color=color)
ax2.tick_params(axis='y', labelcolor=color)

# Format the x-axis to display the time nicely
ax1.xaxis.set_major_locator(mdates.AutoDateLocator())
ax1.xaxis.set_major_formatter(mdates.DateFormatter('%H:%M'))

fig.tight_layout() # To ensure there's no overlap in labels
plt.title('Temperature and Mean Magnetic Field Strength over Time')
plt.show()

```



Removing Unwanted Phases:

Weird Jump at the End Caused by Data Overwrite

```
In [ ]: # Ensure there's more than one record after deduplication
if len(df) > 1:
    # Calculate the absolute difference in 'B_mean' from one point to the next
    df['B_mean_diff'] = df['B_mean'].diff().abs()
    # Compute the mean and standard deviation of the differences, ignoring NaNs
    mean_diff = df['B_mean_diff'].mean()
    std_diff = df['B_mean_diff'].std()
    # Calculate the maximum difference
    max_diff = df['B_mean_diff'].max()
else:
    mean_diff = std_diff = max_diff = np.nan

# Display the diagnostic information
print(f"Number of records: {len(df)}")
print(f"Mean difference: {mean_diff}")
print(f"Standard deviation of difference: {std_diff}")
print(f"Maximum difference: {max_diff}")
```

Number of records: 4153

Mean difference: 30.484308658128924

Standard deviation of difference: 140.07961617476147

Maximum difference: 6640.212574129757

```
In [ ]: # Adjust the threshold
threshold = mean_diff * 100

# Find the index where the dramatic jump occurs
jump_indices = df.index[df['B_mean_diff'] > threshold]

if not jump_indices.empty:
    # Get the first index where the jump occurs
    first_jump_index = jump_indices[0]

    # Remove all data after the jump
    df = df[df.index < first_jump_index]
```

Define the start and end times of the eclipse in local time (EDT)

```
start_of_eclipse_local = edt.localize(datetime(2024, 4, 8, 13, 51)) # Start of partial
eclipse at 1:51 PM EDT -> 13:51 EDT -> 17:51 UTC end_of_eclipse_local =
edt.localize(datetime(2024, 4, 8, 16, 30)) # End of partial eclipse at 4:30 PM EDT -
>16:30 EDT -> 20:30 UTC
```

Convert these times to UTC

```
start_of_eclipse_utc = start_of_eclipse_local.astimezone(utc) end_of_eclipse_utc =
end_of_eclipse_local.astimezone(utc)
```

Uncertainty Calculations for Magnetometer

If the file imported was smr.60s, most of the items in the data frame are an average of many readings. For instance, each magnetometer reading used in the plots was the average of many samples from the magnetometer taken in under a second. To incorporate this into our uncertainty, we use the number of samples to calculate the standard error, not just the number of data points.

```
In [ ]: # Calibration Phase (EDT):
# Start: Monday, April 8, 2024 1:54:55.499 PM
# End: Monday, April 8, 2024 1:57:27.270 PM

# Normalize times - ensure the time range is correctly set
normalize_begin = pd.to_datetime('2024-04-08 13:54:55') # EDT: 13:57:00+00:00
normalize_end = pd.to_datetime('2024-04-08 13:57:27') # EDT: 13:59:00+00:00

# Ensure df_normalize is not just a view but a copy of the slice for safe op
df_normalize = df[(df.index >= normalize_begin) & (df.index <= normalize_end)]
```

```
In [ ]: # Assumed data frame has columns 'BxnT', 'BynT', 'BznT', 'nsamp'
# First, calculate the standard deviation for BxnT, BynT, BznT during calibration
std_dev_Bx = df_normalize['BxnT'].std()
std_dev_By = df_normalize['BynT'].std()
std_dev_Bz = df_normalize['BznT'].std()

# Get the mean number of samples (assuming it might vary)
mean_nsamp = df_normalize['nsamp'].mean()

# Adjust the standard deviations for the number of samples
adjusted_std_dev_Bx = std_dev_Bx / np.sqrt(mean_nsamp)
adjusted_std_dev_By = std_dev_By / np.sqrt(mean_nsamp)
adjusted_std_dev_Bz = std_dev_Bz / np.sqrt(mean_nsamp)

# Combine with manufacturer's specified noise at max sample rate (e.g., 15 r/s)
# Here I assume a conservative approach by using the max specified value if not provided
df_normalize['noise_Bx'] = max(adjusted_std_dev_Bx, 15e-9)
df_normalize['noise_By'] = max(adjusted_std_dev_By, 15e-9)
df_normalize['noise_Bz'] = max(adjusted_std_dev_Bz, 15e-9)

# Output the results
print(f"Noise for Bx: {df_normalize['noise_Bx'].mean()} nT")
print(f"Noise for By: {df_normalize['noise_By'].mean()} nT")
print(f"Noise for Bz: {df_normalize['noise_Bz'].mean()} nT")
```

Noise for Bx: 24.51826031891511 nT

Noise for By: 25.65365521330735 nT

Noise for Bz: 74.03100296217711 nT

```

In [ ]: # Calculate partial derivatives for each component
df_normalize['partial_Bx'] = df_normalize['BxnT'] / df_normalize['B_mean']
df_normalize['partial_By'] = df_normalize['BynT'] / df_normalize['B_mean']
df_normalize['partial_Bz'] = df_normalize['BznT'] / df_normalize['B_mean']

# Calculate total uncertainty for each row
df_normalize['total_uncertainty'] = np.sqrt(
    (df_normalize['partial_Bx'] * df_normalize['noise_Bx'])**2 +
    (df_normalize['partial_By'] * df_normalize['noise_By'])**2 +
    (df_normalize['partial_Bz'] * df_normalize['noise_Bz'])**2
)

# Display the DataFrame with calculated uncertainties
#df_normalize[['B_mean', 'total_uncertainty']]

method_0_uncertainty = df_normalize['total_uncertainty'].mean()
print(method_0_uncertainty)

```

68.45656193277243

Method 1: Insanely High:

Error Propagation for Magnetic Field Measurements

The uncertainty in the magnetic field measurements (B_{mean}) is calculated using error propagation from the measurements of the magnetic field components B_x , B_y , and B_z . Each component is measured multiple times (n_{samp} samples), allowing us to compute the Standard Error of the Mean (SEM) for each component. The SEM provides a measure of the accuracy of the mean magnetic field measurements and is computed as:

$$\text{SEM} = \frac{\sigma}{\sqrt{n}}$$

where σ is the standard deviation of the component measurements and n is the number of samples for that component.

Calculating Uncertainties

The combined uncertainty for the magnetic field magnitude (B_{mean}) is calculated using the formula:

$$\sigma_{B_{\text{mean}}} = \sqrt{\left(\frac{\partial B_{\text{mean}}}{\partial B_x} \sigma_{B_x}\right)^2 + \left(\frac{\partial B_{\text{mean}}}{\partial B_y} \sigma_{B_y}\right)^2 + \left(\frac{\partial B_{\text{mean}}}{\partial B_z} \sigma_{B_z}\right)^2}$$

where the partial derivatives are the ratios of each component to the total magnetic field magnitude, and σ_{B_x} , σ_{B_y} , σ_{B_z} are the SEMs of each component.

```
In [ ]: # Calculate standard deviations for each magnetic field component
std_BxnT = df_normalize['BxnT'].std()
std_BynT = df_normalize['BynT'].std()
std_BznT = df_normalize['BznT'].std()

# Calculate the standard errors using nsamp
df_normalize.loc[:, 'sem_BxnT'] = std_BxnT / np.sqrt(df_normalize['nsamp'])
df_normalize.loc[:, 'sem_BynT'] = std_BynT / np.sqrt(df_normalize['nsamp'])
df_normalize.loc[:, 'sem_BznT'] = std_BznT / np.sqrt(df_normalize['nsamp'])

# Calculate the uncertainty in B_mean using propagation of uncertainty
df_normalize.loc[:, 'sigma_B_mean'] = np.sqrt(
    (df_normalize['BxnT'] / df_normalize['B_mean'] * df_normalize['sem_BxnT']
    (df_normalize['BynT'] / df_normalize['B_mean'] * df_normalize['sem_BynT']
    (df_normalize['BznT'] / df_normalize['B_mean'] * df_normalize['sem_BznT']
))

# Calculate the mean uncertainty of B_mean for the normalization period
mean_sigma_B_mean = df_normalize['sigma_B_mean'].mean()
mean_BxnT = df_normalize['sigma_B_mean'].mean()

print(f"Mean uncertainty in B_mean during normalization: {mean_sigma_B_mean:}
```

Mean uncertainty in B_mean during normalization: 68.49

Method 2: Flimsy but more accurate

Calibration Phase (EDT)

Time Range

- **Start:** Monday, April 8, 2024 1:54:55.499 PM
- **End:** Monday, April 8, 2024 1:57:27.270 PM

Data Normalization

To ensure accurate measurements, the data is normalized by focusing on a specific timeframe during which no external magnetic field fluctuations were expected. This timeframe allows us to establish a baseline for magnetic field readings.

Time Normalization

Normalized times are set to ensure the data falls within the expected quiet period:

- **Begin:** 1:54:55 PM EDT (equivalent to 17:57 UTC)
- **End:** 1:57:27 PM EDT (equivalent to 17:59 UTC)

Calculations

The mean and standard deviation for the magnetic field components during this period are calculated as follows:

- **Mean B_x :** $\{\text{mean}\}(B_x)$
- **Mean B_y :** $\{\text{mean}\}(B_y)$
- **Mean B_z :** $\{\text{mean}\}(B_z)$
- **Standard Deviation B_x :** σ_{B_x}
- **Standard Deviation B_y :** σ_{B_y}
- **Standard Deviation B_z :** σ_{B_z}

The total number of samples (`nsamp`) collected during this calibration phase is also aggregated to compute a more accurate standard error of the mean (SEM) for each component, thus refining the uncertainty estimation for the mean magnetic field strength:

$$\text{SEM}_{B_x} = \frac{\sigma_{B_x}}{\sqrt{\text{total nsamps}}}$$

$$\text{SEM}_{B_y} = \frac{\sigma_{B_y}}{\sqrt{\text{total nsamps}}}$$

$$\text{SEM}_{B_z} = \frac{\sigma_{B_z}}{\sqrt{\text{total nsamps}}}$$

Error Propagation

Using these SEM values, the uncertainty in the mean magnetic field strength (B_{mean}) is recalculated:

$$\sigma_{B_{\text{mean}}} = \sqrt{\left(\frac{\text{mean}(B_x)}{B_{\text{mean}}} \times \text{SEM}_{B_x}\right)^2 + \left(\frac{\text{mean}(B_y)}{B_{\text{mean}}} \times \text{SEM}_{B_y}\right)^2 + \left(\frac{\text{mean}(B_z)}{B_{\text{mean}}} \times \text{SEM}_{B_z}\right)^2}$$

Results

- Total number of samples during normalization: `total_nsamps`
- Adjusted uncertainty in B_{mean} : $\sigma_{B_{\text{mean}}}$

```
In [ ]: # Calibration Phase (EDT):  
# Start:Monday, April 8, 2024 1:54:55.499 PM  
# End: Monday, April 8, 2024 1:57:27.270 PM  
  
# Normalize times - ensure the time range is correctly set  
normalize_begin = pd.to_datetime('2024-04-08 13:54:55') # EDT: 13:57:00+00:00
```



```

normalize_end = pd.to_datetime('2024-04-08 13:57:27') # EDT: 13:59:00+00:00'

# Correct the filtering to include data between start and end times
df_normalize = df[(df.index >= normalize_begin) & (df.index <= normalize_end)]

mean_B_mean = df_normalize['B_mean'].mean()

mean_BxnT = df_normalize['BxnT'].mean()
mean_BynT = df_normalize['BynT'].mean()
mean_BznT = df_normalize['BznT'].mean()

std_BxnT = df_normalize['BxnT'].std()
#sem_BxnT = std_BxnT / np.sqrt(len(df_normalize))

std_BynT = df_normalize['BynT'].std()
#sem_BynT = std_BynT / np.sqrt(len(df_normalize))

std_BznT = df_normalize['BznT'].std()
#sem_BznT = std_BznT / np.sqrt(len(df_normalize))

# Get the total number of samples used in the calibration period
total_nsamps = df_normalize['nsamp'].sum()

# Calculate the SEM using the total_nsamps
sem_BxnT = std_BxnT / np.sqrt(total_nsamps)
sem_BynT = std_BynT / np.sqrt(total_nsamps)
sem_BznT = std_BznT / np.sqrt(total_nsamps)

# Now recalculate the uncertainty in B_mean with the new SEM values
partial_BxnT = mean_BxnT / mean_B_mean
partial_BynT = mean_BynT / mean_B_mean
partial_BznT = mean_BznT / mean_B_mean

low_sigma_B_mean = np.sqrt(
    (partial_BxnT * sem_BxnT) ** 2 +
    (partial_BynT * sem_BynT) ** 2 +
    (partial_BznT * sem_BznT) ** 2
)

print(f"Total nsamps during normalization: {total_nsamps}")
print(f"Uncertainty in B_mean with adjusted nsamps: {low_sigma_B_mean:.2f}")

print(f"Mean BxnT: {mean_BxnT:.2f}, Std Dev: {std_BxnT:.2f}, SEM: {sem_BxnT:.2f}")
print(f"Mean BynT: {mean_BynT:.2f}, Std Dev: {std_BynT:.2f}, SEM: {sem_BynT:.2f}")
print(f"Mean BznT: {mean_BznT:.2f}, Std Dev: {std_BznT:.2f}, SEM: {sem_BznT:.2f}")

```

Total nsamps during normalization: 5446
 Uncertainty in B_mean with adjusted nsamps: 5.61
 Mean BxnT: 16799.69, Std Dev: 148.23, SEM: 2.01
 Mean BynT: 7069.25, Std Dev: 155.09, SEM: 2.10
 Mean BznT: -41296.08, Std Dev: 447.57, SEM: 6.06

Method Comparison

When comparing the two methods of calculating the uncertainty in B_{mean} , where one uses the total number of samples across the calibration period (aggregate method) and the other uses the number of samples per measurement (per measurement method), there are key differences to consider:

Aggregate Method

- **Pros:**
 - **Pooling Samples:** By pooling all samples from the calibration phase, this method leverages a larger denominator in the SEM calculation, potentially leading to a smaller SEM and hence a lower calculated uncertainty in B_{mean} . This is advantageous when measurements are consistent across the time period.
 - **Robust to Outliers:** Aggregating samples can mitigate the impact of any outlier measurements on the calculated SEM, as the variability is averaged out over more samples.
- **Cons:**
 - **Assumes Homogeneity:** This method assumes that the measurements are homogeneous and stable across the calibration period, which might not be true if there were subtle variations or drifts in the magnetometer readings.

Per Measurement Method

- **Pros:**
 - **Measurement-Specific Uncertainty:** Calculates uncertainty specific to each magnetic field reading, allowing for adjustments based on the specific conditions or stability at each measurement time. This could be more accurate if the measurement conditions vary over time.
 - **Sensitive to Variations:** This method can highlight variations in measurement precision or instrument stability that might be averaged out in the aggregate method.
- **Cons:**
 - **Potentially Higher Uncertainties:** As each measurement's uncertainty is calculated independently, variations in individual readings can lead to higher overall uncertainties if some measurements had fewer samples or higher variability.
 - **More Sensitive to Outliers:** Individual outliers have a more pronounced effect

on the calculated uncertainty for their respective timestamps.

Which is Better?

- **Data Homogeneity:**
 - If the data across the calibration period is homogeneous and stable, the **aggregate method** might be preferable due to its potential for lower overall uncertainty.
- **Data Variability:**
 - If there is significant variability in measurement conditions or sensor stability across different times, the **per measurement method** provides a more tailored approach, acknowledging that each measurement's context affects its reliability.

In conclusion, the choice between these methods should be guided by the nature of the experimental data and the specific requirements of the analysis. If accuracy and specificity are critical and the conditions under which data are collected vary, the per measurement method is superior. Conversely, for overall trends and where conditions are stable, the aggregate method may suffice. Further statistical tests or a detailed examination of the data's underlying characteristics might also help in deciding the most appropriate method for a given set of data.

Post-Calibration Data

We no longer need the calibration data since we calculated the uncertainty.

```
In [ ]: calibration_end = pd.to_datetime('2024-04-08 14:05') # 14:05:00+00:00 EDT ->
df = df[df.index >= calibration_end]
```

Initial Plot Part 2

```

In [ ]: import matplotlib.pyplot as plt
import matplotlib.dates as mdates

# Create the figure and the two axes
fig, ax1 = plt.subplots(figsize=(10, 6))
ax2 = ax1.twinx() # Second y-axis for the magnetic field strength

# Plot temperature on ax1
color = 'tab:red'
ax1.set_xlabel('Time on April 8, 2024 (EDT)')
ax1.set_ylabel('Temperature (°C)', color=color)
ax1.plot(df.index, df['ctemp'], color=color, label='Temperature', linewidth=2)
ax1.tick_params(axis='y', labelcolor=color)

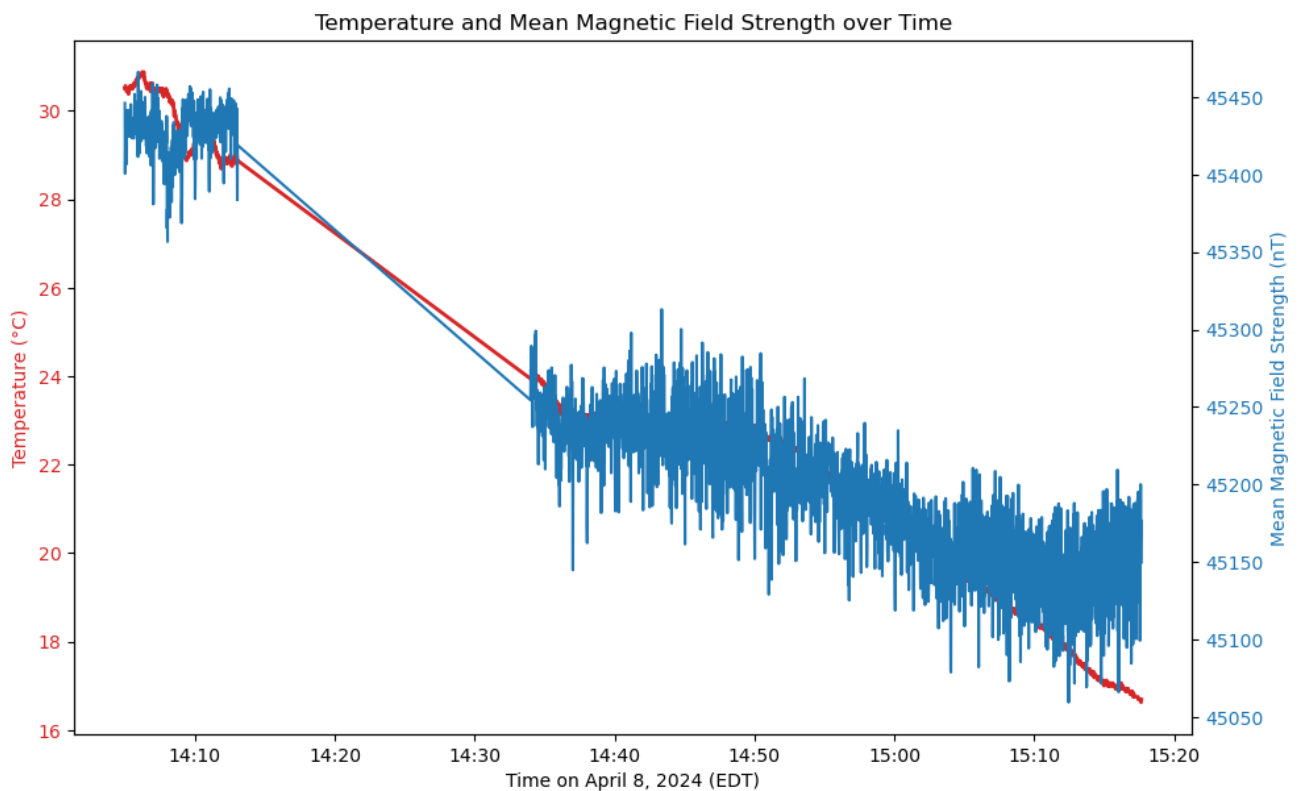
# Plot magnetic field on ax2
color = 'tab:blue'
ax2.set_ylabel('Mean Magnetic Field Strength (nT)', color=color)
ax2.plot(df.index, df['B_mean'], color=color, label='Magnetic Field', zorder=2)
ax2.tick_params(axis='y', labelcolor=color)

# Format the x-axis to display the time nicely
ax1.xaxis.set_major_locator(mdates.AutoDateLocator())
ax1.xaxis.set_major_formatter(mdates.DateFormatter('%H:%M'))

fig.tight_layout()
plt.title('Temperature and Mean Magnetic Field Strength over Time')

plt.show()

```



Analyzing Correlation

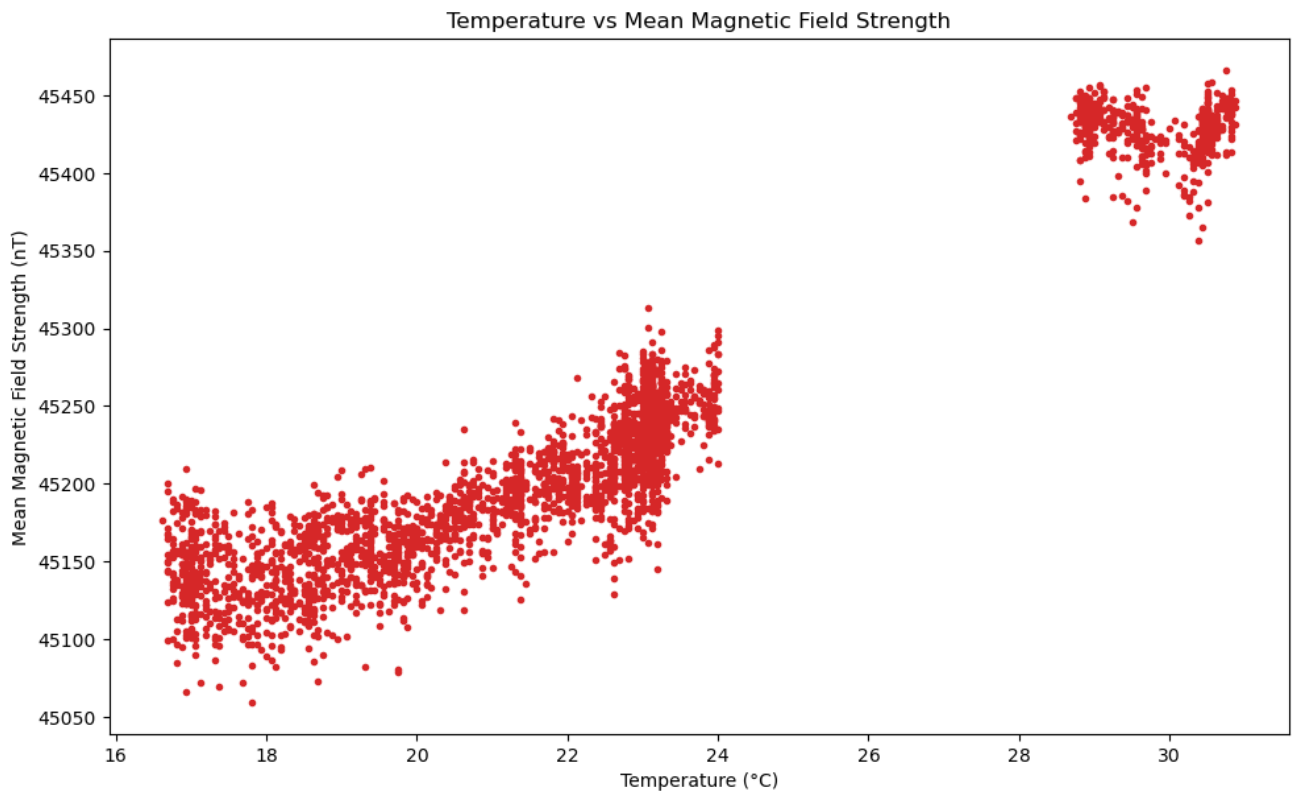
There seems to be a downward trend in both the temp and the mag field. Let's take a closer look.

```
In [ ]: import matplotlib.pyplot as plt
import matplotlib.dates as mdates

# Plotting Temperature ('ctemp') over time
fig, ax1 = plt.subplots(figsize=(10, 6))

color = 'tab:red'
ax1.set_ylabel('Mean Magnetic Field Strength (nT)')
ax1.set_xlabel('Temperature (°C)')
ax1.scatter(df['ctemp'], df['B_mean'], color=color, marker='.')
ax1.tick_params(axis='y')

fig.tight_layout() # To ensure there's no overlap in labels
plt.title('Temperature vs Mean Magnetic Field Strength')
plt.show()
```



```
In [ ]: def validate_inputs(data_x, data_y, d_data_x, d_data_y):
        """
        Validates the input data for errors or anomalies.

        Args:
        data_x (np.ndarray): The x data.
```

`data_y (np.ndarray)`: The y data.
`d_data_x (np.ndarray)`: The uncertainties in the x data.
`d_data_y (np.ndarray)`: The uncertainties in the y data.

Returns:

`bool`: True if all checks pass, False otherwise.

"""

Check for empty arrays

```
if data_x.size == 0:
    print("data_x is empty.")
    return False
if data_y.size == 0:
    print("data_y is empty.")
    return False
if d_data_x.size == 0:
    print("d_data_x is empty.")
    return False
if d_data_y.size == 0:
    print("d_data_y is empty.")
    return False
```

Check for any NaNs or Infs

```
if np.isnan(data_x).any() or np.isinf(data_x).any():
    print("data_x contains NaNs or Infs.")
    return False
if np.isnan(data_y).any() or np.isinf(data_y).any():
    print("data_y contains NaNs or Infs.")
    return False
if np.isnan(d_data_x).any() or np.isinf(d_data_x).any():
    print("d_data_x contains NaNs or Infs.")
    return False
if np.isnan(d_data_y).any() or np.isinf(d_data_y).any():
    print("d_data_y contains NaNs or Infs.")
    return False
```

Check for non-positive uncertainties

```
if (d_data_x <= 0).any():
    print("d_data_x contains non-positive uncertainties.")
    return False
if (d_data_y <= 0).any():
    print("d_data_y contains non-positive uncertainties.")
    return False
```

Check that the arrays are all the same length

```
if not (data_x.size == data_y.size == d_data_x.size == d_data_y.size):
    print("Input arrays are not the same length.")
    return False
```

```
print("All input checks passed.")
return True
```

```

In [ ]: df['d_ctype'] = 0.25
df['d_B_mean'] = low_sigma_B_mean #method_0_uncertainty

# Your data inputs
data_x = df['ctype']
data_y = df['B_mean']
d_data_x = df['d_ctype']
d_data_y = df['d_B_mean']

print(np.mean(data_x), np.mean(data_y))
print(np.mean(d_data_x), np.mean(d_data_y))

# Validate inputs
if not validate_inputs(data_x, data_y, d_data_x, d_data_y):
    # If the validation fails, handle the error appropriately
    print("Input validation failed. Please check the data.")
else:
    # Proceed with the analysis since the validation passed
    # ...
    pass

```

22.373484 45229.38432945836

0.25 5.60820933190145

All input checks passed.

```

In [ ]: tol = 0.001 # Tolerance for iteration convergence

# Function to iteratively refine parameters with error propagation
# Select the method here to switch between fitting functions
method = log_fit # Change to power_fit as needed

params, cov = get_params(data_x, data_y, d_data_x, d_data_y, method, tol)
print("Params:", params)
print("Cov:", cov)

plt.figure(figsize=(10, 6)) # Increased figure size for better visualization
ax = plt.subplot()

# Plot data with error bars
ax.errorbar(data_x, data_y, xerr=d_data_x, yerr=d_data_y, fmt='.', label='Observed Data')

# Generate a smooth line for the fitted model
x_min, x_max = np.min(data_x), np.max(data_x)
x_smooth = np.linspace(x_min, x_max, 500) # Smooth x values for plotting the fit

# Linear fit expects parameters m and b
fitted_data = method(x_smooth, params['A'], params['B'])
fit_label = f"Linear Fit: {np.round(params['A'],4)} * x + {np.round(params['B'],4)}"

y_err = prediction_band(x_smooth, cov, params)

ax.scatter(x_smooth, fitted_data, label=f'Linear Fit: {params["A"]:.4f} * x + {params["B"]:.4f}')
#ax.fill_between(x_smooth, fitted_data - y_err, fitted_data + y_err, color='lightcoral')

# Corrected labels for the plot
ax.set_xlabel('Temperature (°C)', fontsize=12) # Correct label for temperature
ax.set_ylabel('Magnetic Field Strength (nT)', fontsize=12) # Correct label for magnetic field strength

# Adjust the title based on the method used
title = 'Linear Fit for Temperature versus Magnetic Field Strength (nT)'

ax.set_title(title, fontsize=14)
ax.legend()

ax.set_ylim([np.min(data_y) - np.ptp(data_y)*0.1, np.max(data_y) + np.ptp(data_y)*0.1])

plt.show()

```

```

/Users/SEELMPC1/anaconda3/envs/new_env/lib/python3.9/site-packages/scipy/optimize/_minpack_py.py:906: OptimizeWarning: Covariance of the parameters could not be estimated

```

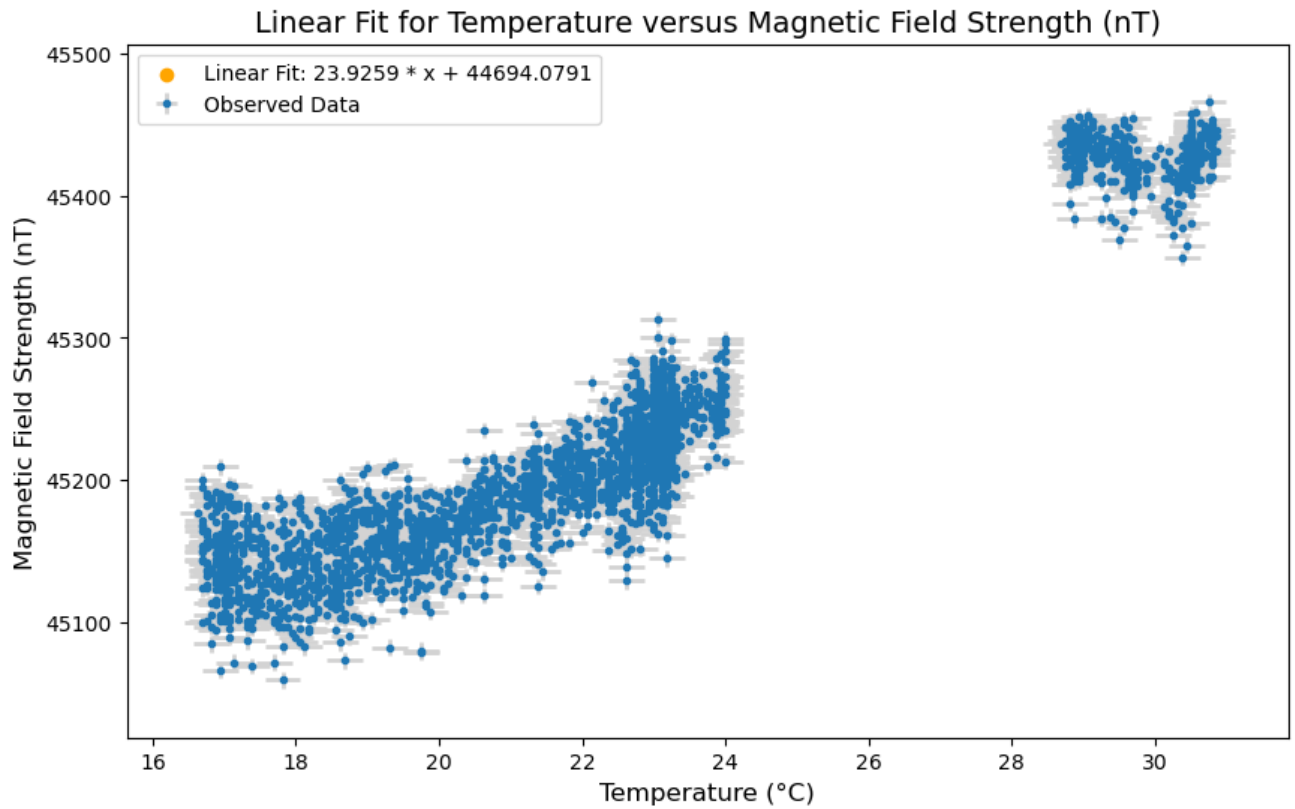
```

    warnings.warn('Covariance of the parameters could not be estimated',
2024-04-15 02:42:52,278 - INFO - Iteration 0: {'A': 23.925875533404373, 'dA': inf, 'B': 44694.079121663985, 'dB': inf}
2024-04-15 02:42:52,283 - INFO - Iteration 1: {'A': 23.925875533404373, 'dA': inf, 'B': 44694.079121663985, 'dB': inf}
2024-04-15 02:42:52,283 - INFO - Convergence reached.

```


Params: {'A': 23.925875533404373, 'dA': inf, 'B': 44694.079121663985, 'dB': i
nf}

Cov: [[inf inf]
[inf inf]]



Magnetic Field vs Time

```

In [ ]: import matplotlib.dates as mdates

# Normalize the time data (convert to total seconds from the start of the data)
time_start = df.index.min()
df['time_seconds'] = (df.index - time_start).total_seconds()

# Data for fitting
data_x = df['time_seconds']
data_y = df['B_mean']

# Assuming you've calculated the uncertainties in your B_mean correctly
d_data_y = df['d_B_mean']

df['low_sigma_mean'] = low_sigma_B_mean

# Avoid zeros in error by setting a minimum threshold
d_data_y = np.maximum(d_data_y, 1e-10)

# Logarithmic fit function
def log_fit(x, a, b):
    return a * np.log(x) + b

# Fit the model
popt, pcov = curve_fit(log_fit, data_x[data_x > 0], data_y[data_x > 0], sigma=d_data_y)

# Calculate the fit line
fit_line = log_fit(data_x, *popt)

# Plotting
plt.figure(figsize=(10, 6))

# Convert time in seconds back to datetime for plotting
times = pd.to_datetime(time_start + pd.to_timedelta(df['time_seconds'], unit='s'))

# Plot the data and the fit
plt.errorbar(times, data_y, yerr=d_data_y, fmt='.', label='Observed Data', color='black')
plt.plot(times, fit_line, label='Logarithmic Fit', color='red')

# Formatting the plot
plt.xlabel('Time')
plt.ylabel('Magnetic Field Strength (nT)')
plt.title('Logarithmic Fit for Time versus Magnetic Field Strength')
plt.legend()

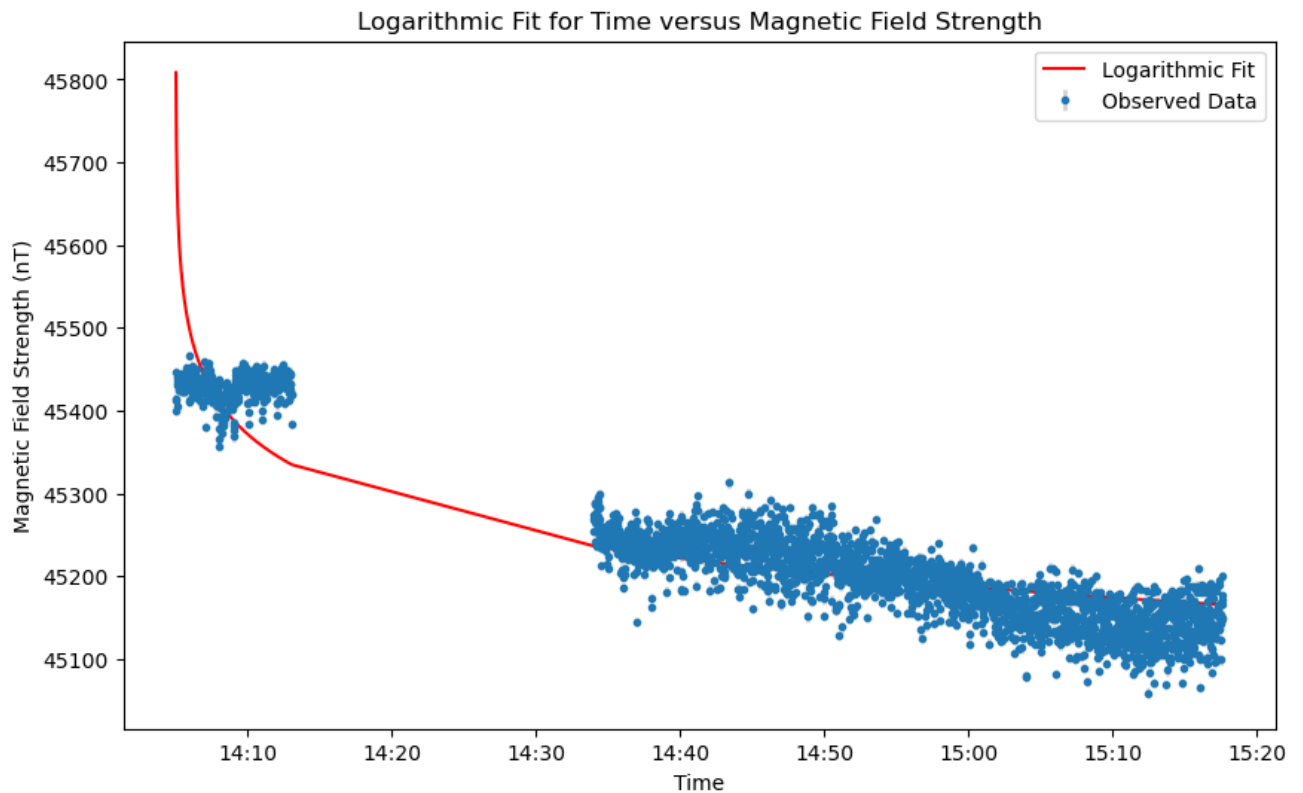
# Set x-axis to show time formatted as hours and minutes
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%H:%M'))

# Display the plot
plt.show()

# Output the parameters
print("Fit Parameters:", pop)
print("Covariance Matrix:", pcov)

```

```
/Users/SEELMPC1/anaconda3/envs/new_env/lib/python3.9/site-packages/pandas/core/arraylike.py:402: RuntimeWarning: divide by zero encountered in log
result = getattr(ufunc, method)(*inputs, **kwargs)
```



```
Fit Parameters: [ -76.84632915 45809.90068486]
```

```
Covariance Matrix: [[ 0.00833223 -0.06295144]
```

```
[-0.06295144  0.4859178  ]]
```

```
In [ ]: # Data for fitting
data_x = df['time_seconds']
data_y = df['B_mean']

df['method_0_uncertainty'] = method_0_uncertainty
# Assuming you've calculated the uncertainties in your B_mean correctly
d_data_y = df['low_sigma_mean'] # method_0_uncertainty

def prediction_band(x, fit_func, params, cov):
    # Predicted values
    fit_y = fit_func(x, *params)

    # Calculate the Jacobian matrix of the fit function at each x with respect to the parameters
    # This assumes a linear fit model. If your model is non-linear, you might need to calculate the Jacobian differently.
    J = np.vstack([x, np.ones_like(x)]).T # For linear model: J[i] = [x[i], 1]

    # Calculate prediction standard deviation: sqrt(J * cov * J^T)
    # We perform a dot product of J with cov, and then dot product of the result with J^T
    pred_var = np.sum(J @ cov * J, axis=1)
    sigma_pred = np.sqrt(pred_var)

    # Student t-value for 2-tail (95%)
```

```

t_val = t.ppf(0.975, len(x) - len(params))

# Prediction interval
interval = t_val * sigma_pred * np.sqrt(1 + 1/len(x) + (x - np.mean(x))**2 / len(x))
lower = fit_y - interval
upper = fit_y + interval
return lower, upper

# Fit the model
popt, pcov = curve_fit(linear_fit, data_x, data_y, sigma=d_data_y, absolute_sigma=True)

# Calculate the fit line
fit_line = linear_fit(data_x, *popt)

# Calculate the prediction band
pred_band_lower, pred_band_upper = prediction_band(data_x, linear_fit, popt, d_data_y)

print("Prediction Band Lower - Min:", np.min(pred_band_lower))
print("Prediction Band Lower - Max:", np.max(pred_band_lower))
print("Prediction Band Upper - Min:", np.min(pred_band_upper))
print("Prediction Band Upper - Max:", np.max(pred_band_upper))

# Fit Parameters and their uncertainties
popt, pcov = curve_fit(linear_fit, data_x, data_y, sigma=d_data_y, absolute_sigma=True)
a = popt[0]
b = popt[1]
sigma_a = np.sqrt(pcov[0, 0])
sigma_b = np.sqrt(pcov[1, 1])

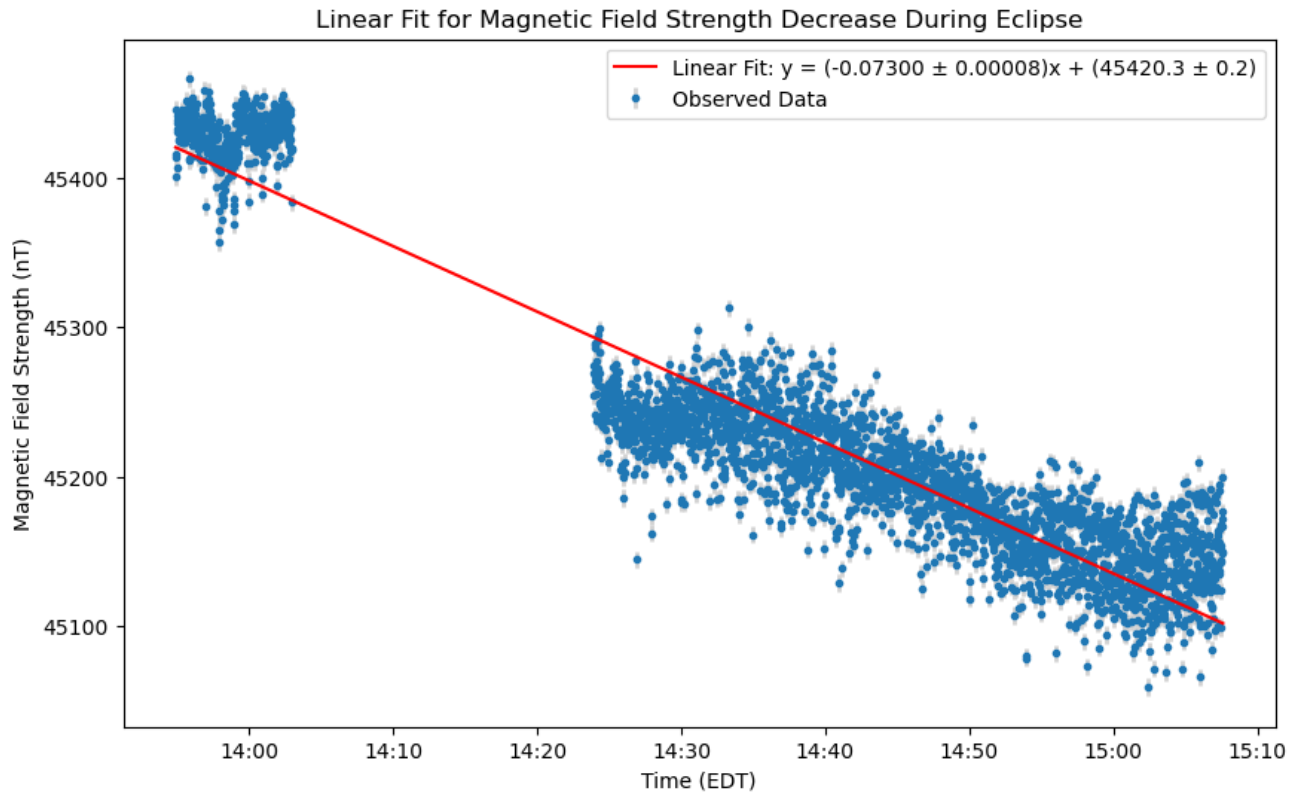
# Correctly formatted label for the linear fit
fit_label = f'Linear Fit: y = ({a:.5f} ± {sigma_a:.5f})x + ({b:.1f} ± {sigma_b:.1f})'
#fit_label = f'Linear Fit: y = ({a:.3f} ± {sigma_a:.3f})x + ({b:.0f} ± {sigma_b:.0f})'

# Printing the parameters and uncertainties for clarity
print("Fit Parameters:", popt)
print("Fit Parameter Uncertainties:", [sigma_a, sigma_b])
print("Covariance Matrix:", pcov)

# Continue with your plotting as before, using `fit_label` for the legend label
plt.figure(figsize=(10, 6))
times = pd.to_datetime(df['time_seconds'], unit='s', origin=pd.Timestamp('2000-01-01T00:00:00'))
plt.errorbar(times, df['B_mean'], yerr=d_data_y, fmt='.', label='Observed Data')
plt.plot(times, fit_line, label=fit_label, color='red', zorder=3)
#plt.fill_between(times, pred_band_lower, pred_band_upper, color='orange', alpha=0.5)
plt.xlabel('Time (EDT)')
plt.ylabel('Magnetic Field Strength (nT)')
plt.title('Linear Fit for Magnetic Field Strength Decrease During Eclipse')
plt.legend()
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%H:%M'))
plt.show()

```

Prediction Band Lower – Min: 45101.68751848492
 Prediction Band Lower – Max: 45419.84074087246
 Prediction Band Upper – Min: 45102.376194100856
 Prediction Band Upper – Max: 45420.77280443224
 Fit Parameters: $[-7.29985180 \times 10^{-2} \quad 4.54203068 \times 10^4]$
 Fit Parameter Uncertainties: $[8.208073433152819 \times 10^{-5}, 0.23746853986505725]$
 Covariance Matrix: $\begin{bmatrix} 6.73724695 \times 10^{-9} & -1.76207914 \times 10^{-5} \\ -1.76207914 \times 10^{-5} & 5.63913074 \times 10^{-2} \end{bmatrix}$



```

In [ ]: # Data for fitting
data_x = df['time_seconds']
data_y = df['B_mean']

df['method_0_uncertainty'] = method_0_uncertainty
# Assuming you've calculated the uncertainties in your B_mean correctly
d_data_y = df['method_0_uncertainty'] # method_0_uncertainty

def prediction_band(x, fit_func, params, cov):
    # Predicted values
    fit_y = fit_func(x, *params)

    # Calculate the Jacobian matrix of the fit function at each x with respect to the parameters
    # This assumes a linear fit model. If your model is non-linear, you might need to calculate the Jacobian manually.
    J = np.vstack([x, np.ones_like(x)]).T # For linear model: J[i] = [x[i], 1]

    # Calculate prediction standard deviation: sqrt(J * cov * J^T)
    # We perform a dot product of J with cov, and then dot product of the result with J^T
    pred_var = np.sum(J @ cov * J, axis=1)
    sigma_pred = np.sqrt(pred_var)

```

```

# Student t-value for 2-tail (95%)
t_val = t.ppf(0.975, len(x) - len(params))

# Prediction interval
interval = t_val * sigma_pred * np.sqrt(1 + 1/len(x) + (x - np.mean(x))**2)
lower = fit_y - interval
upper = fit_y + interval
return lower, upper

# Fit the model
popt, pcov = curve_fit(linear_fit, data_x, data_y, sigma=d_data_y, absolute_sigma=True)

# Calculate the fit line
fit_line = linear_fit(data_x, *popt)

# Calculate the prediction band
pred_band_lower, pred_band_upper = prediction_band(data_x, linear_fit, popt, d_data_y)

print("Prediction Band Lower - Min:", np.min(pred_band_lower))
print("Prediction Band Lower - Max:", np.max(pred_band_lower))
print("Prediction Band Upper - Min:", np.min(pred_band_upper))
print("Prediction Band Upper - Max:", np.max(pred_band_upper))

# Fit Parameters and their uncertainties
popt, pcov = curve_fit(linear_fit, data_x, data_y, sigma=d_data_y, absolute_sigma=True)
a = popt[0]
b = popt[1]
sigma_a = np.sqrt(pcov[0, 0])
sigma_b = np.sqrt(pcov[1, 1])

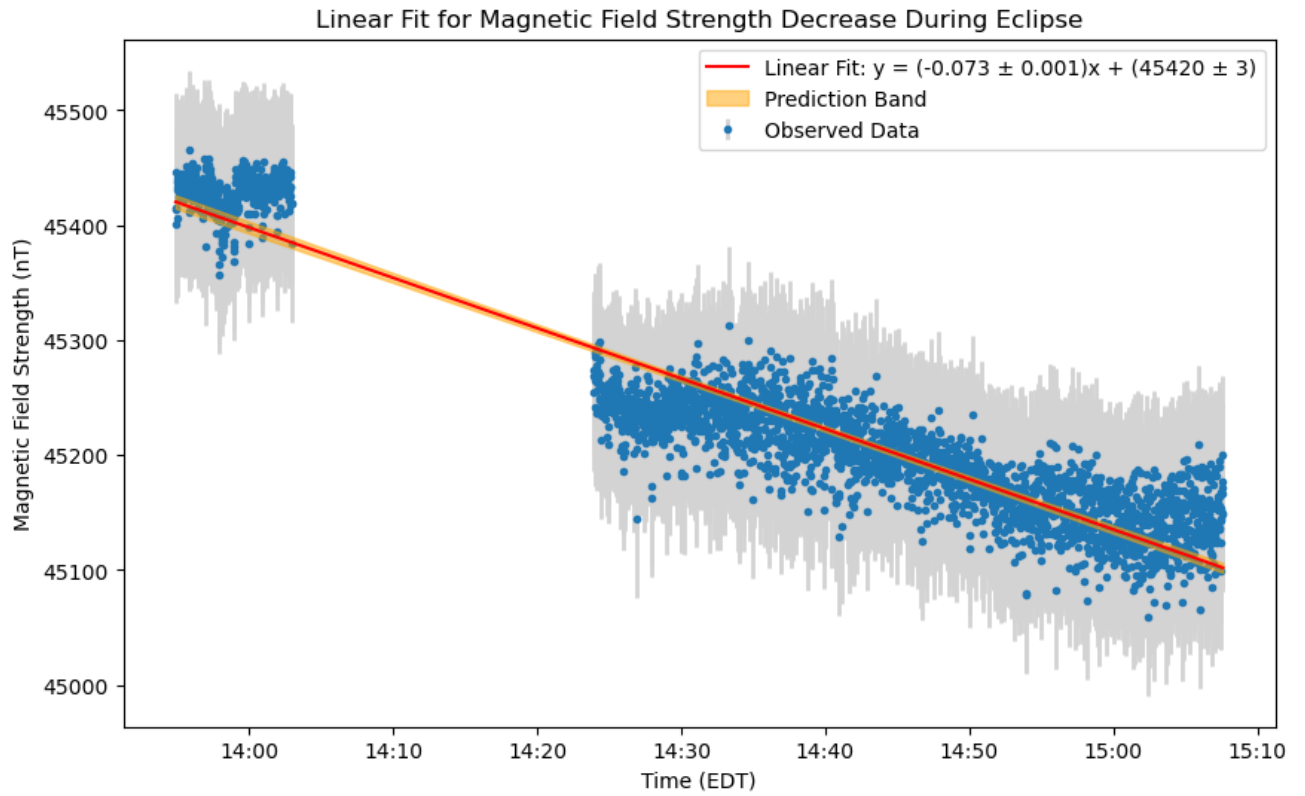
# Correctly formatted label for the linear fit
fit_label = f'Linear Fit: y = ({a:.3f} ± {sigma_a:.3f})x + ({b:.0f} ± {sigma_b:.0f})'

# Printing the parameters and uncertainties for clarity
print("Fit Parameters:", popt)
print("Fit Parameter Uncertainties:", [sigma_a, sigma_b])
print("Covariance Matrix:", pcov)

# Continue with your plotting as before, using `fit_label` for the legend label
plt.figure(figsize=(10, 6))
times = pd.to_datetime(df['time_seconds'], unit='s', origin=pd.Timestamp('2000-01-01T00:00:00'))
plt.errorbar(times, df['B_mean'], yerr=d_data_y, fmt='.', label='Observed Data')
plt.plot(times, fit_line, label=fit_label, color='red', zorder=3)
plt.fill_between(times, pred_band_lower, pred_band_upper, color='orange', alpha=0.5)
plt.xlabel('Time (EDT)')
plt.ylabel('Magnetic Field Strength (nT)')
plt.title('Linear Fit for Magnetic Field Strength Decrease During Eclipse')
plt.legend()
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%H:%M'))
plt.show()

```

Prediction Band Lower – Min: 45097.828698706544
Prediction Band Lower – Max: 45414.618160089536
Prediction Band Upper – Min: 45106.23501273135
Prediction Band Upper – Max: 45425.995386944414
Fit Parameters: $[-7.29985183 \times 10^{-2} \quad 4.54203068 \times 10^4]$
Fit Parameter Uncertainties: $[0.0010019179043086045, 2.8986579515685533]$
Covariance Matrix: $\begin{bmatrix} 1.00383949 \times 10^{-6} & -2.62547090 \times 10^{-3} \\ -2.62547090 \times 10^{-3} & 8.40221792 \times 10^0 \end{bmatrix}$



```

In [ ]: # Data for fitting
data_x = df['time_seconds']
data_y = df['imu_ctemp']

d_data_y = df['d_ctemp']

# Fit the model
popt, pcov = curve_fit(linear_fit, data_x, data_y, sigma=d_data_y, absolute_

# Calculate the fit line
fit_line = linear_fit(data_x, *popt)

# Calculate the prediction band
pred_band_lower, pred_band_upper = prediction_band(data_x, linear_fit, popt,

print("Prediction Band Lower - Min:", np.min(pred_band_lower))
print("Prediction Band Lower - Max:", np.max(pred_band_lower))
print("Prediction Band Upper - Min:", np.min(pred_band_upper))
print("Prediction Band Upper - Max:", np.max(pred_band_upper))

# Fit Parameters and their uncertainties
popt, pcov = curve_fit(linear_fit, data_x, data_y, sigma=d_data_y, absolute_
a = popt[0]
b = popt[1]
sigma_a = np.sqrt(pcov[0, 0])
sigma_b = np.sqrt(pcov[1, 1])

# Correctly formatted label for the linear fit
fit_label = f'Linear Fit: y = ({a:.6f} ± {sigma_a:.6f})x + ({b:.3f} ± {sigma
#fit_label = f'Linear Fit: y = ({a:.3f} ± {sigma_a:.3f})x + ({b:.0f})'

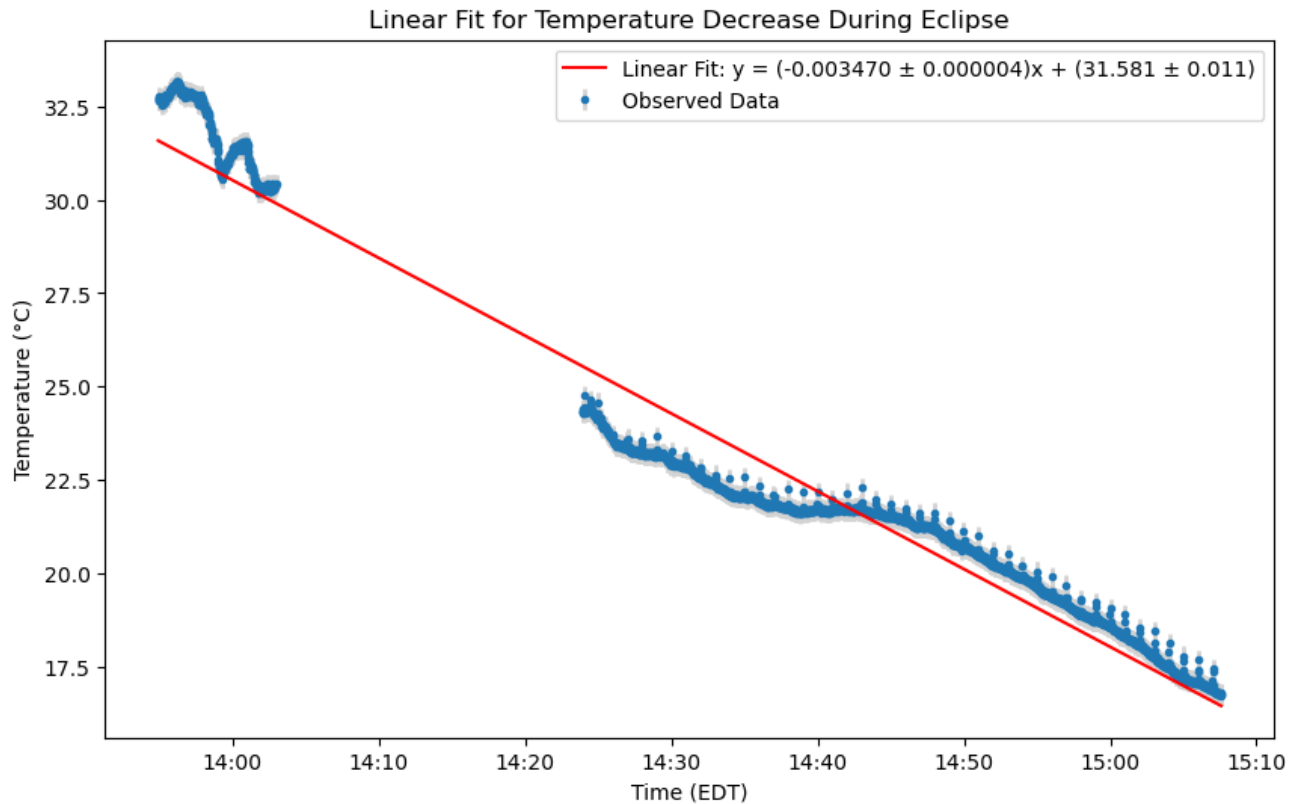
# Printing the parameters and uncertainties for clarity
print("Fit Parameters:", popt)
print("Fit Parameter Uncertainties:", [sigma_a, sigma_b])
print("Covariance Matrix:", pcov)

# Continue with your plotting as before, using `fit_label` for the legend la

plt.figure(figsize=(10, 6))
times = pd.to_datetime(df['time_seconds'], unit='s', origin=pd.Timestamp('20
plt.errorbar(times, data_y, yerr=d_data_y, fmt='.', label='Observed Data', e
plt.plot(times, fit_line, label=fit_label, color='red', zorder=3)
#plt.fill_between(times, pred_band_lower, pred_band_upper, color='orange', a
plt.xlabel('Time (EDT)')
plt.ylabel('Temperature (°C)')
plt.title('Linear Fit for Temperature Decrease During Eclipse')
plt.legend()
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%H:%M'))
plt.show()

```


Prediction Band Lower – Min: 16.438034331464234
 Prediction Band Lower – Max: 31.56052592608973
 Prediction Band Upper – Min: 16.468733776404196
 Prediction Band Upper – Max: 31.602074999623063
 Fit Parameters: [-3.46969058e-03 3.15813005e+01]
 Fit Parameter Uncertainties: [3.6589549107526355e-06, 0.01058575643379455]
 Covariance Matrix: [[1.33879510e-11 -3.50152359e-08]
 [-3.50152359e-08 1.12058239e-04]]



In []: `df['time_seconds']`

Out []: `dateTime`

2024-04-08 14:05:00.030511	0.000000
2024-04-08 14:05:01.049388	1.018877
2024-04-08 14:05:02.068549	2.038038
2024-04-08 14:05:03.088216	3.057705
2024-04-08 14:05:04.107470	4.076959
...	
2024-04-08 15:17:36.036444	4356.005933
2024-04-08 15:17:37.039094	4357.008583
2024-04-08 15:17:38.042316	4358.011805
2024-04-08 15:17:39.045422	4359.014911
2024-04-08 15:17:40.049387	4360.018876

Name: time_seconds, Length: 3052, dtype: float64

In []:

TO-DO:

1. Look at readings of background magnetic field from earlier and compare quantitatively with the other shift
2. Use models
3. Figure out what electromagnetic bullshit was supposed to happen

TIME??

Coordinates via Google Maps:

41°22'55.9"N 81°04'17.6"W

In []: